JTC1/SC22/WG21 mailing list 09.2026 overview



Vasil Pashov
Senior Engineer, Man Group

Presentation to ”C++ and programming languages” working group at Bulgarian Institute for Standardization, Bulgaria on Thursday 29th October 2026. For educational purposes only.

Intended for C++ professionals and students only

All extracts provided in this presentation are illustrative examples – for information only

© Man 2026

Opinions expressed are those of the Speaker as the date of the conference and are subject to change. The content is not intended to constitute advice nor a recommendation or opinion regarding any investment or strategy. Views concerning outlook or other non-historical matters are based on current indicators and expectations at the date of the conference and are subject to risks and uncertainties that may cause actual results to differ materially.

Official working group page



© Man 2026

Man Group

Global active investment management firm, founded in 1783

Systematic trading

One of the largest public investment management firms listed on LSE

Offices around the globe, London, Boston, Sofia among others

© Man 2026



Man

Database engineering at Man Group

Handles billions of rows with low-latency reads/writes

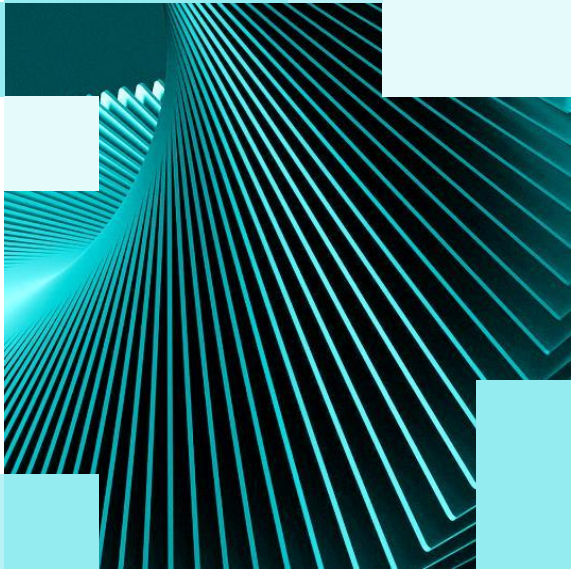
Open Source

Serverless

Python API

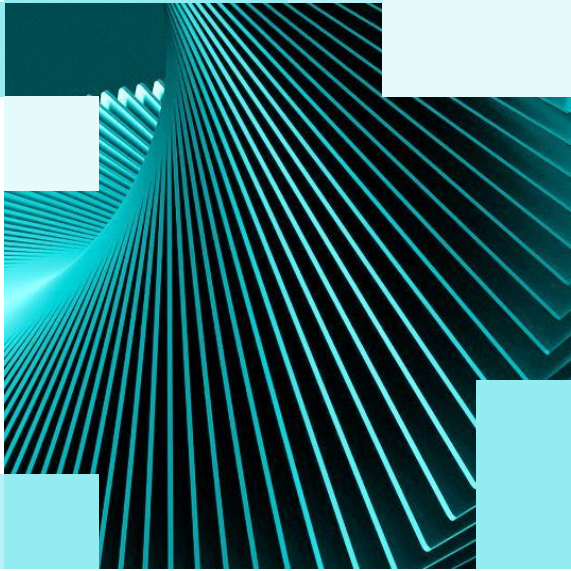
© Man 2026

JTC1/SC22/WG21 mailing list 09.2026 overview



© Man 2026

P2806R5 – do expressions



© Man 2026

P2806R5 - do expressions

- Resembles immediately invoked lambda
- New keyword: do_return – to return from the expression
- Use return/break/continue to return from the outer scope

© Man 2026

For illustrative purposes only.

P2806R5 - do expressions

```
1 int x = std::rand();
2 const bool is_negative = do {
3     if(x < 0) {
4         do_return true;
5     } else {
6         do_return false;
7     }
8 }
```

© Man 2026

For illustrative purposes only.

P2806R5 - do expressions

```
1 int f(int x) {
2     const int y = do {
3         if(x == 0) return 5; // returns from f
4         else do_return 10; // "returns" from the do expression
5     };
6     std::cout<<"After the expression\n";
7     return y * 2;
8 }
```

© Man 2026

For illustrative purposes only.

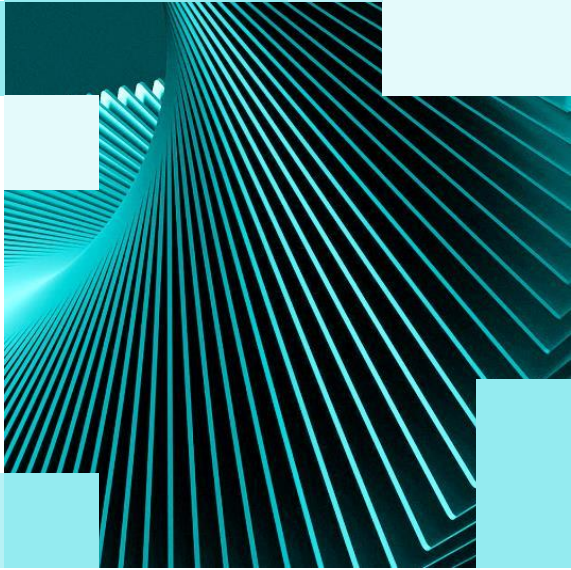
P2806R5 - do expressions

```
1 for(int x = 0; i < n; ++i) {
2     const int x = do {
3         if(i < 5) continue;
4         else if(i > 10) break;
5         do_return i;
6     }
7 }
```

© Man 2026

For illustrative purposes only.

P2688R6 - Pattern Matching with match and case



© Man 2026

P2688R6 Pattern Matching with match and case

- Allow pattern matching on (almost) anything in C++
- Can be statement or expression
- Cases are do_expressions
- Match expression must handle every possible scenario

© Man 2026

For illustrative purposes only.

P2688R6 Pattern Matching with match and case

Match a value

```
int f(int n) {
    auto res = match (n) {
        case 5 => return 10;
        case _ => 2;
    };
    return res;
}
```

© Man 2026

For illustrative purposes only.

P2688R6 Pattern Matching with match and case

Unpacking

```
int main() {
    std::variant<int, std::pair<std::optional<int>, int>> v;
    auto res = match (v) {
        case { int i }                => i;
        case { [{ int a }, auto b] }  => a + b;  // pair whose optional has a value
        case { [{}, int b] }          => b;      // pair whose optional is empty
    };
}
```

© Man 2026

For illustrative purposes only.

P2688R6 Pattern Matching with match and case

Shorthand

```
1 int main() {
2     std::optional<int> v;
3     if(case {int a} = v) {
4         std::cout<<"Optional has value\n";
5     }
6 }
```

© Man 2026

For illustrative purposes only.

P2688R6 Pattern Matching with match and case

Dynamic cast

```
1 void draw(Shape& shape) {
2   match (shape) {
3     case Circle& circle       => draw_circle(circle);
4     case Triangle& triangle   => draw_triangle(triangle);
5     case Rectangle& rectangle => draw_rectangle(rectangle);
6     case _                    => draw_unknown(shape);
7   }
8 }
```

© Man 2026

For illustrative purposes only.

P2688R6 Pattern Matching with match and case

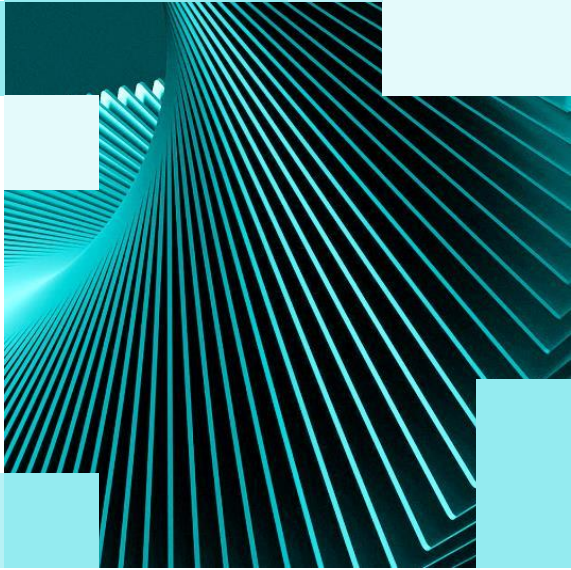
Guards

```
1    std::optional<int> n{10};
2    match(n) {
3        case {} => std::cout<<"empty\n";
4        case {int a} if (a > 5) => std::cout<<"Larger than 5\n";
5        case {int} => std::cout<<"fallback\n";
6    }
```

© Man 2026

For illustrative purposes only.

P2561R3 - A control flow operator



© Man 2026

P2561R3 A control flow operator – current state

```cpp
auto foo(int i) -> std::expected<int, E>;
auto bar(int i) -> std::expected<int, E>;

auto strcat(int i) -> std::expected<std::string, E>
{
    auto f = foo(i);
    if (not f) return std::unexpected(f.error());

    auto b = bar(i);
    if (not b) return std::unexpected(b.error());

    return std::format("{}{}", *f, *b);
}
```

© Man 2026

For illustrative purposes only.

P2561R3 A control flow operator

- Uses pattern matching and do_expression
- The operator either returns a value or returns from the enclosing function
- The enclosing function must have return type implementing std::try_traits

© Man 2026

For illustrative purposes only.

P2561R3 A control flow operator – desugaring

```
match (expr) -> decltype(auto) {
    case auto&& __e => do -> decltype(auto) {
        using ETraits = std::try_traits<std::remove_cvref_t<decltype(__e)>>;
        using RTraits = std::try_traits<
            typename [: return_type_of(std::meta::current_function()) :]>;

        if (not ETraits::should_continue(__e)) {
            return RTraits::from_break(ETraits::extract_break(FWD(__e)));
        }

        do_return ETraits::extract_continue(FWD(__e));
    };
}
```

© Man 2026

For illustrative purposes only.

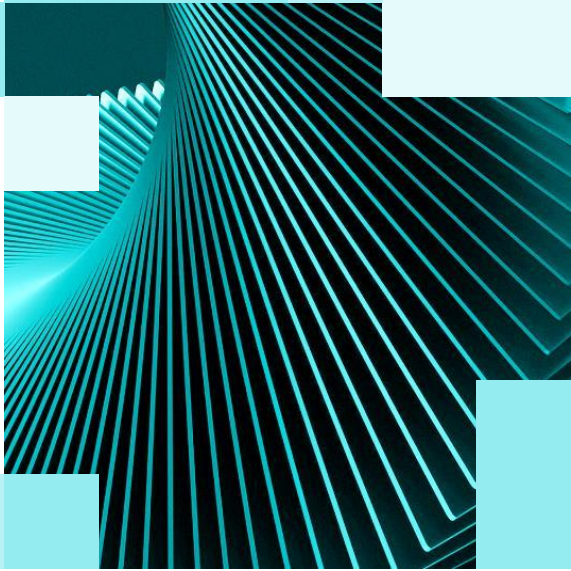
P2561R3 A control flow operator – after

```
1 auto foo(int i) -> std::expected<int, E>;
2 auto bar(int i) -> std::expected<int, E>;
3
4 auto strcat(int i) -> std::expected<std::string, E>
5 {
6     auto f = foo(i).try?;
7     auto b = bar(i).try?;
8     return std::format("{}{}", f, b);
9 }
```

© Man 2026

For illustrative purposes only.

P4040R2 Case ranges



© Man 2026

P4040R2 Case ranges

- Already accepted into the C29 standard
- C++ aims for compatibility with C
- Can be expressed with match

© Man 2026

For illustrative purposes only.

P4040R2 Case ranges

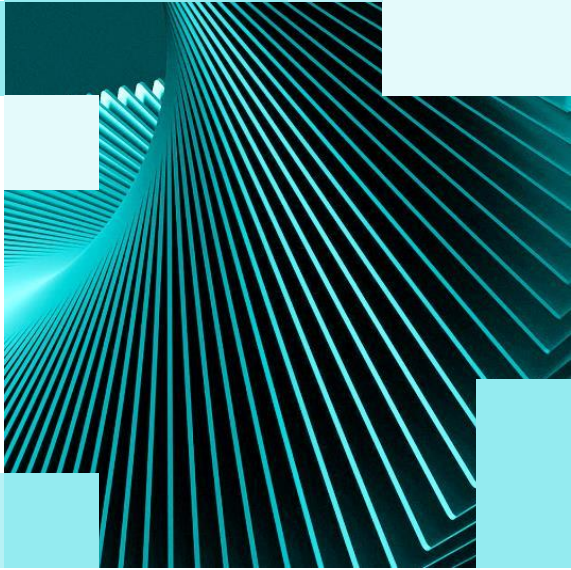
```
switch (next_char) {
  case '0':
  case '1':
  case '2':
  case '3':
  case '4':
  case '5':
  case '6':
  case '7':
  case '8':
  case '9': // ...
}
```

```
switch (next_char) {
  case '0' ... '9': // ...
}
```

© Man 2026

For illustrative purposes only.

P4380R0 Token Sequence Injection



© Man 2026

P4380R0 Token Sequence Injection

- **C++ 26 static reflection give little way of generating code via define_aggregate**
- **Token sequence: ^^{}**
  - e.g. ^^{int, if, 5, return}
  - Random access
- **Token interpolation \e()**
  - ^^{\e(std::vector<int>)}
  - std::vector<int> is a single token, not the 6 tokens the lexer would parse: std, ::, vector, <, int, >>
- **Injection**
  - std::meta::queue_injection
  - on_complete(r) - invoked when the class r becomes complete
  - on_template_defined - invoked when the class template r is defined (before any specialization happens)
  - inject_members - invoked right before the } when the class r is about to be complete. Prior special members definition

© Man 2026

For illustrative purposes only.

```cpp
struct WithSetters {
    consteval auto inject_members(std::meta::info c) {
        auto ctx = std::meta::access_context::unchecked();
        auto result = ^^{
            public:
        };
        for(auto member : nonstatic_data_members_of(c, ctx)) {
            result += generate_setter(member);
        }
        return result;
    }

    consteval std::meta::token_sequence generate_setter(std::meta::info member) {
        auto member_name = identifier_of(member);
        auto setter_name = id("set_", member_name);
        auto param_name = id("new_", member_name);
        return ^^{
            template<typename T>
            void \(setter_name) (T&& \(param_name)) {
                this->\(member) = std::forward<T>(\(param_name));
            }
        };
    }
};

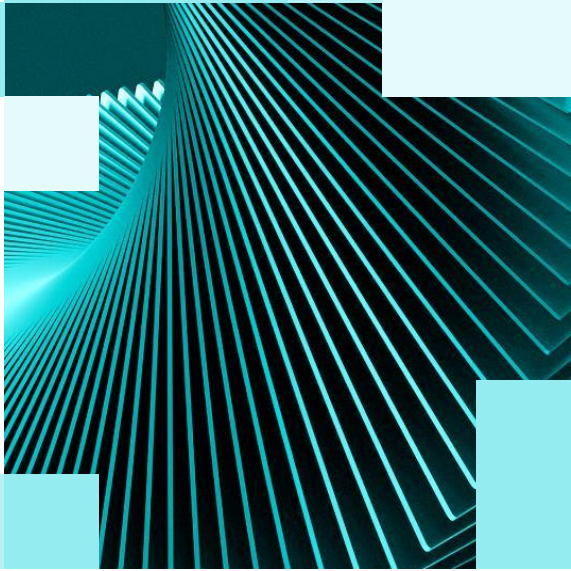
struct [[=WithSetters{}]] S {
    int a;
    std::vector<int> b;
};

int main() {
    S s;
    s.set_a(10);
    s.set_b(std::vector<int>{1, 2, 3});
    std::println("{}", s.a);
    std::println("{}", s.b);
}
```

© Man 2026

For illustrative purposes only.

P3951R2 String Interpolation Objects



© Man 2026

P3951R2 String Interpolation Objects - example

```
1 std::string verb();
2
3 auto example(std::string const& name, std::string relation) {
4     auto tmpl = t"Hello, my name is {name:?}. You killed my {relation}. Prepare to {verb()}.";
5     std::println(tmpl);
6     std::println(t"Hello, my name is {name:?}. You killed my {relation}. Prepare to {verb()}.");
7     return tmpl; // THIS IS NOT SAFE
8 }
```

© Man 2026

For illustrative purposes only.

P3951R2 String Interpolation Objects - desugaring

```cpp
struct Template {
    static consteval auto fmt() -> char const* {
        return "Hello, my name is {:?}. You killed my {}. Prepare to {}.";
    }

    static consteval auto num_interpolations() -> size_t { return 3; }

    static consteval auto string(size_t i) -> char const* {
        constexpr char const* data[] = {
            "Hello, my name is ",
            ". You killed my ",
            ". Prepare to ",
            "."
        };
        return data[i];
    }

    static consteval auto interpolation(size_t i) -> std::interpolation {
        constexpr std::interpolation data[] = {
            {"name",     "{:?}", 0, 1},
            {"relation", "{}",   1, 1},
            {"verb()",   "{}",   2, 1},
        };
        return data[i];
    }

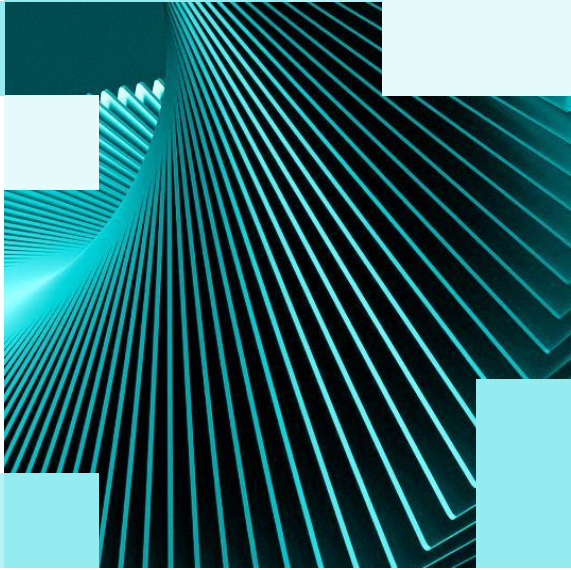
    constexpr auto exprs() const -> Template const& { return *this; }

    std::string const& _0;   // name       (decltype((name))   -> std::string const&)
    std::string&       _1;   // relation   (decltype((relation)) -> std::string&)
    std::string        _2;   // verb()     (prvalue             -> std::string)
};
```

© Man 2026

For illustrative purposes only.

P2034R9 Partially Mutable Lambda Captures



© Man 2026

P2034R9 Partially Mutable Lambda Captures

Current state – global modifiers

```
1 auto f = [n](int x) { return x + n; };
2
3 struct __lambda_f {
4     int n;
5     int operator()(int x) const { return x + n; }
6 };
```

```
1 auto f = [n](int x) mutable { n += x; return n; };
2
3 struct __lambda_f {
4     int n;
5     int operator()(int x) { n += x; return n; }
6 };
```

© Man 2026

For illustrative purposes only.

P2034R9 Partially Mutable Lambda Captures

Proposal

```cpp
auto L = [a, // Existing behavior
          r&, // Existing behavior
          mutable m,
          const c,
          const& cr,
          mutable buf = std::vector<int>{},
          const id = 42]() const {};

struct __lambda_L {
    int a;
    int& r;
    mutable int m;
    const int c;
    const int& cr;
    mutable std::vector<int> buf;
    const int id;

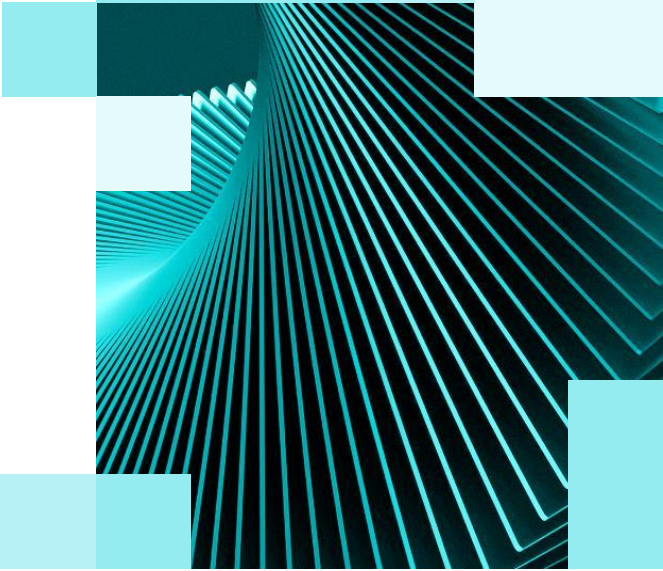
    __lambda_L(int a_, int m_, int c_, int& r_, const int& cr_)
        : a(a_), m(m_), c(c_), r(r_), cr(cr_), buf(std::vector<int>{}), id(42) {}

    void operator()() const {}
};
```

© Man 2026

For illustrative purposes only.

P4341R0 Another take on non-transient constexpr allocation



© Man 2026

P4341R0 Another take on non-transient constexpr allocation

Current state

```cpp
constexpr auto f() {
    // Can have dynamic allocations in constexpr function
    std::vector<int> v;
    v.push_back(5);
    v.push_back(3);
    return v;
}

constexpr auto g() {
    // Can call function with "dynamic" allocation in constexpr context
    constexpr auto sz = f().size();
    std::array<int, sz> arr;
    // Can copy the "dynamic" memory
    std::ranges::copy(f(), arr.data());
    return arr;
}


int main() {
    // Data compute in a constexpr context can reach the runtime via some hoops.
    constexpr auto arr = g();
    // Not allowed. The memory cannot outlive the expression. It must die
    // by the ;
    constexpr auto vec = f();
}
```

© Man 2026

For illustrative purposes only.

P4341R0 Another take on non-transient constexpr allocation

Problem – no deep constness

```
1 // Note: this is not valid C++
2 constexpr std::unique_ptr<std::unique_ptr<int>> ptr{std::make_unique<int>(5)};
3 // Not allowed ptr is const
4 ptr.reset(std::make_unique<std::shared_ptr<int>>{std::make_unique<int>{6}});
5 // Pointers don't propagate constness.
6 // The inner pointer was allocated at compile time.
7 (*ptr).reset(std::make_unique<int>{6});
8
```

© Man 2026

For illustrative purposes only.

P4341R0 Another take on non-transient constexpr allocation

3 approaches

- **[P0784R5] More constexpr containers – trust the user**
- **[P1974R0] Non-transient constexpr allocation using propconst – new type qualifier**
- **[P4341R0]**
  - New specifier immutable_if_constexpr(bool)
  - For a pointer P, check all paths leading to it
  - Path is **blessed** if all paths in the graph are either const or immutable_if_constexpr(true)

© Man 2026

For illustrative purposes only.

P4341R0 Another take on non-transient constexpr allocation

Immutable

```
1 struct H1 {
2     immutable_if_constexpr int const* owner;
3     int const*                        alias;
4     constexpr ~H1() { delete owner; }
5 };
6
7 constexpr H1 h1 = [] { int* p = new int(42); return H1{p, p}; }();
8 static_assert(*h1.alias == 42);
```

© Man 2026

For illustrative purposes only.

P4341R0 Another take on non-transient constexpr allocation

Mutable at runtime, no immutable_if_constexpr

```
1 struct H2 {
2     int const* owner;
3     int const* alias;
4     constexpr ~H2() { delete owner; }
5 };
6
7 constexpr H2 h2 = [] { int* p = new int(42); return H2{p, p}; }();
8 static_assert(*h2.alias == 42); // Not immutable (constexpr)
9 *h2.owner = 24; // OK mutable at runtime
```

© Man 2026

For illustrative purposes only.

P4341R0 Another take on non-transient constexpr allocation

One mutable one blessed

```
1 struct H3 {
2     int*       owner;
3     int const* alias;
4     constexpr ~H3() { delete owner; }
5 };
6
7 constexpr H3 h3 = [] { int* p = new int(42); return H3{p, p}; }();
8 static_assert(*h3.alias == 42); // Not immutable
9 *h3.owner = 24; // Mutable at runtime
```

© Man 2026

For illustrative purposes only.

P4341R0 Another take on non-transient constexpr allocation

Ill formed – one immutable_if_constexpr and one mutable

```
1 // Ill formed
2 struct H4 {
3     immutable_if_constexpr int const* owner;
4     int*                              alias;
5     constexpr ~H4() { delete owner; }
6 };
7 constexpr H4 h4 = [] { int* p = new int(42); return H4{p, p}; }();
```

© Man 2026

For illustrative purposes only.

P4341R0 Another take on non-transient constexpr allocation

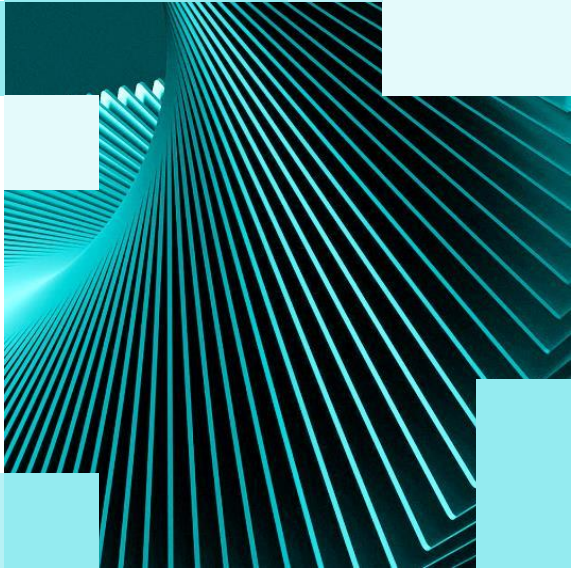
UB

```
1 struct H5 {
2     immutable_if_constexpr int* owner;    // marked, but not const
3     constexpr ~H5() { delete owner; }
4 };
5
6 constexpr H5 h5 = [] { return H5{new int(42)}; }();
7 static_assert(*h5.owner == 42);   // the int is immutable
8
9 void w() { *h5.owner = 0; }       // compiles, but UB: writes to immutable storage
```

© Man 2026

For illustrative purposes only.

P2719R8 Type-aware allocation and deallocation functions



© Man 2026

P2719R8 Type-aware allocation and deallocation functions

- Allow for custom operator new/delete per type/concept
- New must be paired with delete in the same scope
- Can block heap allocation by adding =delete on new
- Defined only inside a class or globally. No namespaces. No ADL

© Man 2026

For illustrative purposes only.

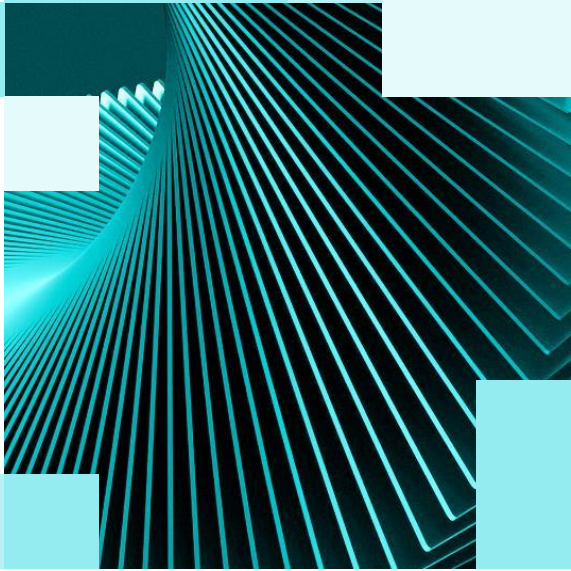
P2719R8 Type-aware allocation and deallocation functions

```
1 template<some_concept C>
2 void* operator new(std::type_identity<C>, std::size_t, std::align_val_t);
3
4 template<some_concept C>
5 void  operator delete(void*);
```

© Man 2026

For illustrative purposes only.

P2952R3 Deduced return types for defaulted operator functions



© Man 2026

P2952R3 Deduced return types for defaulted operator functions

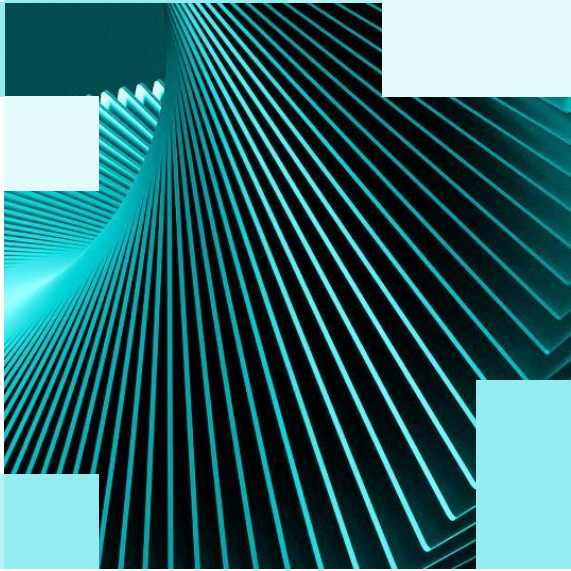
```cpp
struct SomeLongClassName {
  SomeLongClassName& operator=(const SomeLongClassName&) = default;
  SomeLongClassName& operator=(SomeLongClassName&&) = default;
  auto operator<=>(const SomeLongClassName&) const = default;
  bool operator==(const SomeLongClassName&) const = default;
  bool operator<(const SomeLongClassName&) const = default;
};
```

```cpp
struct SomeLongClassName {
  auto& operator=(const SomeLongClassName&) = default;
  auto& operator=(SomeLongClassName&&) = default;
  auto operator<=>(const SomeLongClassName&) const = default;
  auto operator==(const SomeLongClassName&) const = default;
  auto operator<(const SomeLongClassName&) const = default;
};
```

© Man 2026

For illustrative purposes only.

P3606R2 On Overload Resolution, Exact Matches and Clever Implementations



© Man 2026

P3606R2 On Overload Resolution, Exact Matches and Clever Implementations

Similarly, the existence of a definition of a variable or function is considered to affect the semantics of the program if the instantiation is necessary to determine the type of an expression in the program, even if the variable or function is not odr-used

© Man 2026

For illustrative purposes only.

P3606R2 On Overload Resolution, Exact Matches and Clever Implementations

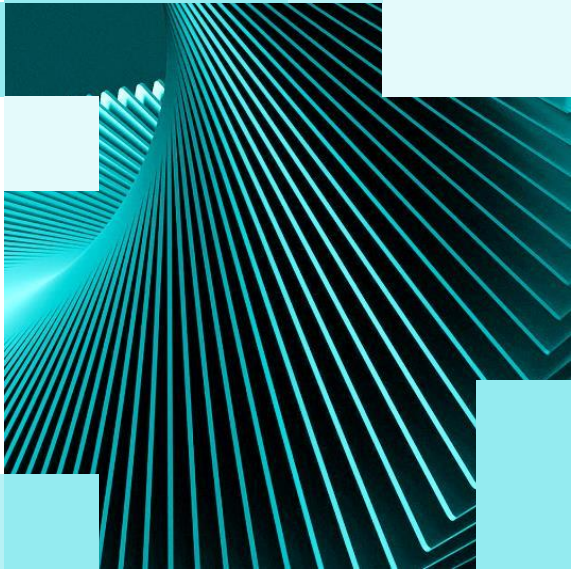
```
1 template<typename T>
2 decltype([] { return T::x;}) f(T);// #1
3
4 void f(int) {} // #2
5
6 int main() {
7   f(0);
8 }
```

- #1 is ill formed. The lambda is not in the immediate context T::x is not valid
- #2 is an exact match
- GCC picks #2 and does not try to evaluate #1

© Man 2026

For illustrative purposes only.

P4235R0 Removing Digraphs



© Man 2026

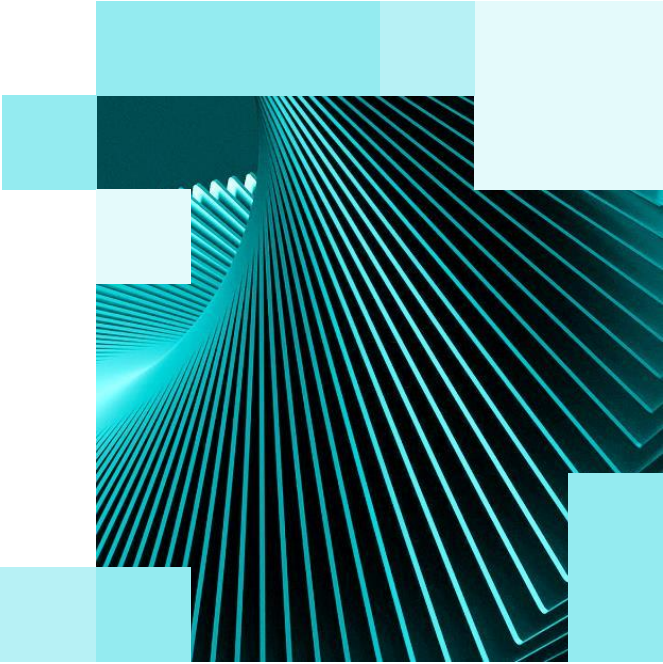
P4235R0 Removing Digraphs

- **Old charsets cannot represent some symbols**
- **Use combination of two symbols to represent them**
  - (. becomes {
  -). becomes }

© Man 2026

For illustrative purposes only.

P4363R0 C++ Contracts compile-time evaluation semantic: static



© Man 2026

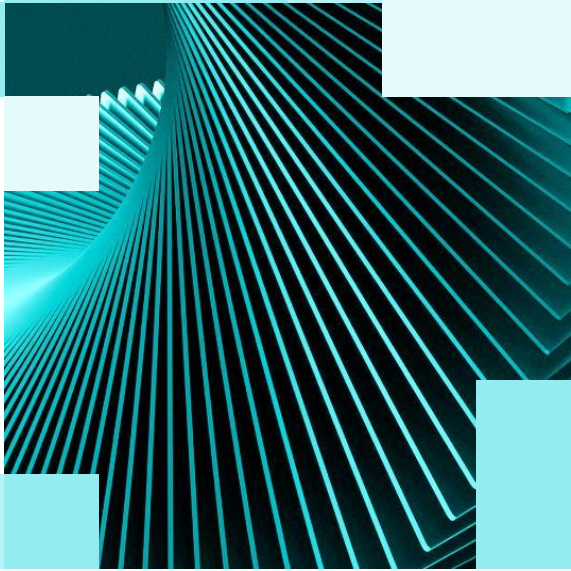
P4363R0 C++ Contracts compile-time evaluation semantic: static

- Static evaluation semantic
- Evaluated only at compile time
- Result: true/false/unknown
- Encouraged to warn on unknown

© Man 2026

For illustrative purposes only.

P4376R0 Do not require module implementation units to import primray module interface implicitly



© Man 2026

P4376R0 Do not require module implementation units to import primray module interface implicitly

- Primary interface unit
  - must reexport every partition
  - Produce BMI
- Interface partition – Produce BMI
- Internal partition – Produce BMI
- Implementation unit
  - automatically imports the interface)
  - No BMI

© Man 2026

For illustrative purposes only.

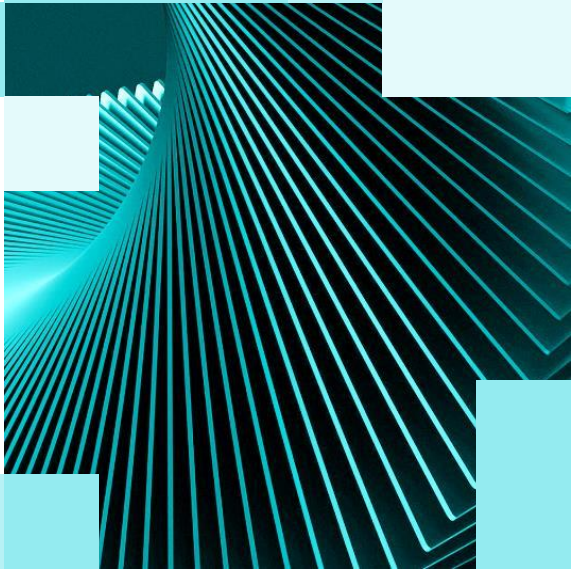
P4376R0 Do not require module implementation units to import primray module interface implicitly

```
1 // m-interfaceA.cppm
2 export module m:interfaceA;
3
4 // m-interfaceB.cppm
5 export module m:interfaceB;
6
7 // Primary
8 export module m;
9 export import :interfaceA;
10 export import :interfaceB;
11
12 // m1.cpp
13 module m;
14 // m2.cpp
15 module m;
```

© Man 2026

For illustrative purposes only.

P4307R0 An Infix Operator and a Keyword Escape for C++



© Man 2026

P4307R0 An Infix Operator and a Keyword Escape for C++

- Use `` (backticks) to escape keywords
- Use ``(backticks) to make function an infix operator

```
1 // Infix operator
2 auto min_val = a `std::min` b;
3 // Escape
4 int `new` = 5;
```

© Man 2026

For illustrative purposes only.

Important Information

This presentation is provided for educational and information purposes only and does not constitute a financial promotion.

This information is communicated and/or distributed by the relevant Man entity identified below (collectively the "Company") subject to the following conditions and restriction in their respective jurisdictions.

Opinions expressed are those of the author and may not be shared by all personnel of Man Group plc ('Man'). These opinions are subject to change without notice, are for information purposes only and do not constitute an offer or invitation to make an investment in any financial instrument or in any product to which the Company and/or its affiliates provides investment advisory or any other financial services. Any organisations, financial instrument or products described in this material are mentioned for reference purposes only which should not be considered a recommendation for their purchase or sale. Neither the Company nor the authors shall be liable to any person for any action taken on the basis of the information provided. Some statements contained in this material concerning goals, strategies, outlook or other non-historical matters may be forward-looking statements and are based on current indicators and expectations. These forward-looking statements speak only as of the date on which they are made, and the Company undertakes no obligation to update or revise any forward-looking statements. These forward-looking statements are subject to risks and uncertainties that may cause actual results to differ materially from those contained in the statements. The Company and/or its affiliates may or may not have a position in any financial instrument mentioned and may or may not be actively trading in any such securities. Unless stated otherwise all information is provided by the Company. **Past performance is not indicative of future results.**

Unless stated otherwise this information is communicated by the relevant entity listed below.

Australia: To the extent this material is distributed in Australia it is communicated by Man Investments Australia Limited ABN 47 002 747 480 AFSL 240581, which is regulated by the Australian Securities & Investments Commission ('ASIC'). This information has been prepared without taking into account anyone's objectives, financial situation or needs.

Austria/Germany/Liechtenstein: To the extent this material is distributed in Austria, Germany and/or Liechtenstein it is communicated by Man (Europe) AG, which is authorised and regulated by the Liechtenstein Financial Market Authority (FMA). Man (Europe) AG is registered in the Principality of Liechtenstein no. FL-0002.420.371-2. Man (Europe) AG is an associated participant in the investor compensation scheme, which is operated by the Deposit Guarantee and Investor Compensation Foundation PCC (FL-0002.039.614-1) and corresponds with EU law. Further information is available on the Foundation's website under www.eas-liechtenstein.li.

European Economic Area: Unless indicated otherwise this material is communicated in the European Economic Area by Man Asset Management (Ireland) Limited ('MAMIL') which is registered in Ireland under company number 250493 and has its registered office at 70 Sir John Rogerson's Quay, Grand Canal Dock, Dublin 2, Ireland. MAMIL is authorised and regulated by the Central Bank of Ireland under number C22513.

Hong Kong SAR: To the extent this material is distributed in Hong Kong SAR, this material is communicated by Man Investments (Hong Kong) Limited and has not been reviewed by the Securities and Futures Commission in Hong Kong.

Japan: To the extent this material is distributed in Japan it is communicated by Man Group Japan Limited, Financial Instruments Business Operator, Director of Kanto Local Finance Bureau (FIBO) No. 624 or other licensed financial instruments business operators for the purpose of providing information on investment strategies, discretionary investment management, and other investment services, etc. provided by Man Group, and is not a disclosure document stipulated by laws and regulations. This material can be communicated only to professional investors or institutional investors, intermediaries, gatekeepers, or consultants who may have sufficient knowledge and experience of related risks.

© Man 2026